

爬蟲開發整理

本文件整理原始討論內容中與資料抓取、落地儲存、資料庫匯入與排程執行相關的部分，作為後續實作資料供應鏈的基礎文件。

目標

爬蟲模組的任務不是直接做策略判斷，而是穩定提供可被策略引擎使用的標準化資料。核心原則如下：

- 抓取與分析解耦
- 原始資料先落地，再匯入資料庫
- 不同資料源透過統一介面封裝
- 排程與執行環境容器化

一、資料來源規劃

資料類型	對應用途	建議來源	備註
OHLCV	均線、乖離率、停損停利	TWSE、TPEX、Yahoo Finance	最基礎且最優先
三大法人買賣超	法人連續結帳、高檔出貨	TWSE、FinMind	需整理欄位名稱
融資融券餘額	籌碼面風險判斷	TWSE	需清理千分位格式
月營收	成長動能衰竭	MOPS	需 POST 與表格解析
季財報 / EPS	基本面出場	MOPS	表格結構較複雜

二、主要端點整理

來源	端點	用途	重點
TWSE	https://www.twse.com.tw/exchangeReport/STOCK_DAY	台股上市 OHLCV	參數為 <code>date</code> (如 20231001) 與 <code>stockNo</code> 。需為西元年。
TPEX	https://www.tpex.org.tw/web/stock/aftertrading/daily_trading_info/st43_result.php	台股上櫃 OHLCV	參數為 <code>d</code> (民國年如 112/10) 與 <code>stk_no</code> 。格式相同。
Yahoo Finance	https://query1.finance.yahoo.com/v8/finance/chart/{symbol}	美股與部分台股	建議直接在 Python 中使用 <code>yfinance</code> 套件, API。
TWSE	https://www.twse.com.tw/exchangeReport/T86	三大法人買賣超	參數 <code>date</code> 。可抓取外資、投信、自營商的每日數據。
TWSE	https://www.twse.com.tw/exchangeReport/MI_MARGN	融資融券餘額	參數 <code>date</code> 與 <code>selectType=STOCK</code> 。需注意千分位格式。
FinMind	https://api.finmindtrade.com/api/v4/data	籌碼替代來源	參數帶入 <code>dataset=TaiwanStockInstitutionalInvestment</code> 。適合不想處理繁瑣官方爬蟲的替代方案。
MOPS	https://mops.twse.com.tw/mops/web/ajax_t05st10_ifrs	月營收	使用 <code>POST</code> 請求。需帶入參數 <code>encodeURIComponent=1, firstin=1, off=1, year, month</code> 。
MOPS	https://mops.twse.com.tw/mops/web/ajax_t163sb04	季財報 / EPS	綜合損益表端點。同樣為 <code>POST</code> 請求, HTML 雜, 需解析。

三、Provider 架構設計

建議以抽象基底類別管理所有資料來源，避免不同網站的欄位格式污染上層服務。

建議職責分工

- providers/base.py
 - 定義 `fetch_ohlcv` 或對應 `fetch` 方法介面 (使用 `@abstractmethod`)
 - 檢查標準欄位是否存在
 - 統一日期與數值型別 (呼叫 `_validate_and_format`)
 - 回傳標準化 DataFrame, 無副作用設計 (不碰資料庫與寫檔)
- providers/twse.py
 - 呼叫 TWSE API
 - 處理民國年、逗號分隔數字與欄位 mapping
- providers/mops.py
 - 發送 `POST` 請求與 Payload, 設定正確 Headers 以避免防爬蟲機制

- 解析 HTML 表格
 - 萃取月營收或 EPS 欄位
- providers/finmind.py
 - 封裝 token 與 dataset 參數
 - 補足官方資料源抓取成本較高的欄位

標準欄位建議

```
["date", "open", "high", "low", "close", "volume"]
```

若是非 OHLCV 資料，可延伸成各自的標準 schema，但仍應遵守欄位命名一致原則。

四、資料落地策略 (JSON Staging Pipeline)

建議採兩段式資料管線 (ELT 模式)：

1. 爬蟲抓取後先無腦落地成 JSON。
2. 匯入程序再讀取本地 JSON 寫入 PostgreSQL。

目錄建議

- data/raw：存放爬下來的原始 JSON
- data/processed：已匯入完成或已清洗完成的檔案

這種設計的優點

- **容錯率極高**：資料庫暫時不可用時，原始資料不會遺失
- **斷點續傳**：可以重跑匯入程序，不必重新爬取，降低防爬蟲封鎖風險
- **Schema 無痛升級**：比較容易包容未來新增或變更的未知欄位
- **兼容 JSONB**：方便之後保留 JSONB 或轉 Parquet 格式的彈性

五、匯入 PostgreSQL 的原則

匯入層建議單獨處理，不要混在 provider 內：

- provider 只負責抓資料與清洗
- insert service 只負責寫入資料庫
- 實作 upsert，避免重複匯入造成主鍵衝突
- 匯入成功後，可搬移 JSON 到 processed 目錄

如果後續要支援更高效能的回測，除了 JSON staging，也可以保留整理後的 Parquet 檔案供分析模組直接讀取。

六、防爬與穩定性處理

1. Request Headers

- TWSE 與 MOPS 請求建議帶瀏覽器 User-Agent。
- 某些端點可能需要 Referer 或 Accept-Language 才較穩定。

2. Throttling

- 官方站點建議加入 3 到 5 秒的隨機延遲。
- 避免短時間密集輪詢造成 IP 被封鎖。

3. 例外處理

- 無資料日要明確跳過，不視為失敗
- 回傳欄位變更時要記錄完整 response 或原始檔
- 資料型態轉換失敗應寫入 log，避免靜默錯誤

七、Docker 執行與排程

建議把爬蟲與排程器打包成獨立容器，和 PostgreSQL 一起交由 Docker Compose 管理。

排程器職責

- 依交易日與時段定時啟動抓取
- 逐一處理 target symbols
- 每檔股票完成後再進行匯入
- 發生錯誤時記錄 log 並繼續處理下一檔

排程建議

- 台股可在每日收盤後執行
- 美股資料可在次日台灣時間早晨批次更新

- 先做全量歷史補資料，再切成每日增量更新

Docker 架構建議

- `postgres`: 資料庫容器
- `crawler_worker`: 負責抓取與匯入，使用 `schedule` 持續監聽
- 掛載 `data` 目錄，保留 JSON 與 log
- 使用 Docker Compose 將兩者串聯，確保 `depends_on`: `postgres`。

八、建議實作順序

1. 先完成 OHLCV provider，因為技術面策略最先依賴它。
2. 再補法人與融資融券資料，支援籌碼面策略。
3. 最後補 MOPS 月營收與 EPS，支援基本面策略。
4. 補上 scheduler 與 Docker Compose，讓每日更新自動化。

九、與策略模組的邊界

爬蟲模組只提供資料，不應包含策略判斷。推薦的分工如下：

- `providers`: 抓資料
- `services/backfill.py`: 補歷史資料
- `services/fetcher.py`: 執行日常抓取
- `services/stock_service.py`: 整合資料供上層使用
- `strategies` 或 `analyzer` 模組: 計算賣出條件

這樣可以避免資料層與策略層耦合，後續新增市場或新增策略時維護成本較低。